



# SISTEMA PREDICTOR DE DEMANDA PARA REDES ELÉCTRICAS

Por Marc Ferrero Fernández

5 de Mayo de 2026

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| 1. INTRODUCCIÓN.....                                                      | 3  |
| 2. FASE 1: COMPRENSIÓN DE NEGOCIO .....                                   | 3  |
| 3. FASE 2: COMPRENSIÓN DE LOS DATOS .....                                 | 4  |
| 3.1 Descripción e inventario de datos.....                                | 5  |
| 3.2.1. Consumo energético: Demanda Real de Energía Peninsular .....       | 5  |
| 3.2.2. Dataset Climatológico: Histórico de Variables Meteorológicas ..... | 6  |
| 3.2.3. Dataset de Contexto: Calendario de días laborales y festivos ....  | 6  |
| 3.3 Análisis de Calidad de los Datos .....                                | 6  |
| 4. FASE 3: PREPARACIÓN DE LOS DATOS.....                                  | 7  |
| 4.1 Preparación de los datos para la Capa Bronce .....                    | 8  |
| 4.2. Ingesta y unificación en la Capa Plata .....                         | 8  |
| 4.2.1. Dataset de Demanda Eléctrica (ESIOS) .....                         | 8  |
| 4.2.2. Dataset de Historial Climático (AEMET) .....                       | 9  |
| 4.2.3. Dataset de Festivos y Calendario Laboral.....                      | 10 |
| 4.3. Integración y Enriquecimiento: Construcción de la Capa Oro ....      | 11 |
| 4.3.1. Cruce de tablas (Joins).....                                       | 11 |
| 4.3.2. Tratamiento de nulos en las temperaturas .....                     | 12 |
| 4.3.3. Ingeniería de Variables (Feature Engineering).....                 | 12 |
| 4.4. Persistencia y Almacenamiento Distribuido (HDFS y Parquet) ....      | 13 |
| 5. FASE 6: DESPLIEGUE .....                                               | 14 |
| 6.1. Conclusiones del Análisis de los Datos .....                         | 15 |
| 6. DICCIONARIO DE DATOS DE LA CAPA ORO .....                              | 16 |

# 1. INTRODUCCIÓN

Hoy en día, con el avance hacia las energías renovables, nos enfrentamos al reto de que la producción de energía cambia constantemente, debido a que depende directamente de factores relacionados con el clima o el calendario de días festivos. Para aprovechar al máximo estas energías limpias, es necesario saber de antemano cuánta electricidad va a necesitar el país en cada momento.

Este informe explica cómo se ha desarrollado un sistema automático para recoger, limpiar y combinar datos sobre el consumo eléctrico, el clima y el calendario. Para llevarlo a cabo, se ha diseñado una estructura mixta que combina el trabajo en la nube (AWS) con el procesamiento en local, organizando la información por capas con la arquitectura Medallón para garantizar que los datos finales sean totalmente limpios y fiables.

## 2. FASE 1: COMPRENSIÓN DE NEGOCIO

Antes de empezar con el desarrollo del sistema, es fundamental tener claro el propósito del proyecto y qué herramientas necesitamos para llevarlo a cabo. A continuación, se detallan las ideas principales que dan forma a este trabajo:

- **Objetivo principal:** Crear un sistema capaz de descargar datos desde diferentes fuentes y agruparlos para estudiar a fondo qué factores influyen en la demanda de electricidad.
- **El problema que resolvemos:** La información útil está muy dispersa (APIs, páginas web, archivos Excel o CSV...), lo que hace que analizarla sea lento y complicado. El objetivo de este proyecto es automatizar la obtención de esos datos en bruto y transformarlos en información limpia y lista para usar.

- Criterios de éxito:
  - Conseguir una descarga inicial de datos que funcione de manera automática usando la Amazon AWS.
  - Lograr construir una tabla final (Capa Oro) que una perfectamente la demanda eléctrica, la temperatura y los días festivos.
  - Crear un panel interactivo que permita visualizar estas tendencias y sacar conclusiones de forma sencilla.

### 3. FASE 2: COMPRENSIÓN DE LOS DATOS

Una vez definidos los objetivos y lo que queremos conseguir, el siguiente paso es revisar a fondo los datos que vamos a utilizar. Para este proyecto, combinamos tres fuentes de información diferentes que se complementan entre sí:

- **Datos de consumo eléctrico:** Es el eje central del proyecto. Usamos la API de **Red Eléctrica de España** para obtener el histórico de la demanda real de electricidad. Estos datos vienen detallados en tramos de 10 minutos, lo que nos permite ver con precisión cómo varía el consumo a lo largo de cada día.
- **Datos Meteorológicos:** El clima influye muchísimo en el uso de la energía debido al uso de calefacciones y aires acondicionados. Para medir este impacto, descargamos desde la web **datosclima.es** los registros históricos de la **AEMET**. Esta fuente incluye información diaria sobre temperaturas, viento y lluvias desde el año 2014 hasta finales de 2025
- **Datos de contexto (Calendario):** El consumo eléctrico varía mucho entre los días laborales y los festivos. Por eso, añadimos un calendario de días festivos para poder entender cómo cambian los hábitos de la gente y de las fábricas según el día de la semana y las fechas especiales.

### 3.1 Descripción e inventario de datos

Para organizar toda la información de manera eficiente y escalable, utilizamos la **arquitectura Medallón**. Este sistema nos permite ordenar los datos en tres capas consecutivas (**Bronze, Plata y Oro**), de forma que la información pasa de estar en bruto tal y como la descargamos, a convertirse en una tabla final totalmente limpia y lista para usar.

A continuación, se detalla la estructura y características técnicas de los tres conjuntos de datos alojados en la capa Bronze de nuestro repositorio de datos:

- **Dataset 1: Demanda Eléctrica (ESIOS)**
  - **Variable Objetivo:** Demanda real de energía medida en Megavatios.
  - **Granularidad:** 10 minutos.
  - **Volumen aproximado:** unos 52.500 registros por año.
  - **Formato original:** JSON.
- **Dataset 2: Histórico Climatológico (AEMET)**
  - **Variables Clave:** Temperatura máxima (°C) y Temperatura mínima (°C)
  - **Granularidad:** Diaria.
  - **Formato original:** Hojas de cálculo (Excel / .xls).
- **Dataset 3: Calendario de días Festivos**
  - **Variables Clave:** Fecha, Nombre del festivo, Ámbito (nacional o local).
  - **Granularidad:** Diaria (solo días festivos).
  - **Formato original:** JSON.

#### 3.2.1. Consumo energético: Demanda Real de Energía Peninsular

- **Origen:** Sistema de Información del Operador del Sistema (ESIOS) de Red Eléctrica de España (REE).
- **Indicador Extraído:** Indicador 1293 ("Demanda real").
- **Granularidad:** Diezminutal (registros fijos cada 10 minutos, sumando 144 observaciones diarias).
- **Volumen Histórico:** Periodo completo desde el 1 de enero de 2014 hasta el 31 de diciembre de 2025 (aproximadamente 630.000 registros).

- **Formato Físico en S3:** Archivos mensuales en formato JSON, organizados bajo un esquema de particionado estándar (**year=YYYY/month=MM/**).

### *3.2.2. Dataset Climatológico: Histórico de Variables Meteorológicas*

- **Origen:** Agencia Estatal de Meteorología (AEMET), extraído de forma agregada a través de la plataforma datosclima.es.
- **Granularidad:** Diaria.
- **Volumen Histórico:** Cobertura total paralela al dataset operativo (2014-2025), estructurado por provincias de la España peninsular.
- **Formato Físico en S3:** Archivos diarios en formato Excel (.xls), particionados por fecha completa en el almacenamiento (**year=YYYY/month=MM/day=DD/**).

### *3.2.3. Dataset de Contexto: Calendario de días laborales y festivos*

- **Origen:** API pública de **Nager.Date** (proveedor global de calendarios festivos).
- **Granularidad:** Diaria (registra únicamente los días del año que albergan alguna festividad).
- **Volumen Histórico:** Rango completo de años del proyecto (2014-2025).
- **Formato Físico en S3:** Archivo anual en formato JSON.

## *3.3 Análisis de Calidad de los Datos*

Al revisar los datos almacenados en la Capa Bronce, se han encontrado tres problemas importantes que marcaron la estrategia a seguir a la hora de transformar los datos para la Capa Plata:

- **Falta de datos en 2013 (ESIOS):** Aunque la documentación de la API decía que había datos disponibles desde 2013, al revisar los archivos comprobamos que la demanda real empezó a registrarse justo el último día de ese año (el 31 de diciembre). **Solución:** Se decidió acotar el estudio y empezar directamente a partir del año 2014.
- **Diferencias en el detalle del tiempo:** El consumo eléctrico viene detallado en tramos de 10 minutos, mientras que los datos del clima de la AEMET se registran una sola vez al día. **Solución:** Al juntar las tablas, se mantuvo el detalle de los 10 minutos del consumo y se repitió el dato del clima de ese día en cada uno de esos tramos, evitando así perder información detallada de la demanda.
- **Conflicto de Huso Horario:** Los datos de consumo (ESIOS) vienen registrados tanto en la hora internacional (UTC) como en la hora de España, mientras que el clima viene solo con la hora de aquí. **Solución:** Para que no haya fallos al combinar las tablas se cambiaron todas las horas al huso horario oficial de la península (Europe/Madrid).

## 4. FASE 3: PREPARACIÓN DE LOS DATOS

Tras revisar a fondo cómo vienen estructurados los datos en crudo y detectar sus fallos, es el momento de prepararlos. El objetivo en esta fase se divide en dos grandes retos: primero, se cargarán, organizarán y estructurarán los datos en crudo en la nube (Capa Bronce); después se corregirán las imperfecciones de los dataset y se unificarán en un mismo dataframe, creando una matriz analítica que saque el máximo potencial del modelo predictor de demanda.

## 4.1 Preparación de los datos para la Capa Bronce

Para que todo este proceso sea transparente, se han dejado los bloques de código que utilizamos dentro de la carpeta **/scripts/** del proyecto. Estos scripts se prepararon de dos maneras diferentes:

- **Funciones Lambda en AWS:** Para el calendario de festivos y la demanda de ESIOS, se prepararon dos funciones Lambda dentro de AWS. Mediante estas funciones, se establece una conexión directa desde AWS a las APIs correspondientes, controlando fallos de red de forma segura y almacenando los archivos JSON estructurados directamente en nuestro bucket de S3 siguiendo la estructura **HIVE**.
- **Estructuración local:** Para el dataset de histórico climático, debido al gran volumen de ficheros y su carencia organizativa, se preparó un script de ejecución local a través de Visual Studio Code. Su función consiste en leer los ficheros, extraer la fecha de sus nombres y organizarlos automáticamente dentro de una estructura de carpetas limpia por año, mes y día (Hive). Una vez que el script ordenó los ficheros, se subió manualmente a AWS.

Con estas acciones, la Capa Bronce quedó lista en la nube para poder empezar con el verdadero trabajo de transformación.

## 4.2. Ingesta y unificación en la Capa Plata

La primera fase consiste en cargar los datos del bucket de AWS S3 en memoria RAM local para iniciar su transformación individual en la capa Plata. Estas fueron las transformaciones que realizaron en cada conjunto de datos:

### 4.2.1. Dataset de Demanda Eléctrica (ESIOS)

La ingesta de esta fuente consistió en unificar los archivos JSON mensuales en un único dataframe histórico de más de 820.000 filas. Las transformaciones más destacadas fueron:

- **Estandarización horaria:** Se convirtió la fecha del documento al huso horario oficial de la península para evitar desfases horarios en los análisis de consumo y en la unificación con el resto de datasets.
- **Generación de columnas de unión:** A partir de la fecha, se crearon dos nuevas columnas: **fecha\_cruce** (con formato de texto **YYYY-MM-DD**) y **hora** (con formato **HH:MM:SS**), necesarias para realizar las uniones posteriores.
- **Supresión de datos innecesarios:** Se descartaron algunas columnas con información poco relevante para el modelo predictor como **geo\_id**, **geo\_name** o **tz\_time**, manteniendo únicamente la fecha, la hora y el valor de la demanda forzado a tipo decimal (float64).

| fecha_cruce | hora     | demanda_mw |
|-------------|----------|------------|
| 2014-01-01  | 00:00:00 | 24546.0    |
| 2014-01-01  | 00:10:00 | 24309.0    |
| 2014-01-01  | 00:20:00 | 24348.0    |
| 2014-01-01  | 00:30:00 | 24321.0    |
| 2014-01-01  | 00:40:00 | 24194.0    |

#### 4.2.2. Dataset de Historial Climático (AEMET)

Esta fuente planteó el mayor número de desafíos debido a sus más de 3,5 millones de registros y a defectos de formato en sus archivos. Las principales correcciones fueron las siguientes:

- **Control de lectura de archivos:** Aunque los más de 4.000 archivos tenían extensión **.xls**, una parte de ellos estaban guardados internamente en formato HTML. Se resolvió programando un bloque **try-except** que intenta abrir el archivo como un Excel normal y, si detecta un fallo, lo lee

automáticamente como HTML. De esta manera se pudo procesar todos los días sin perder información.

- **Limpieza de filas y cabeceras:** Las cabeceras venían mal organizadas o con nombres poco claros, por lo que se sustituyeron por las etiquetas reales de la AEMET. También se eliminaron filas de cabeceras duplicadas surgidas al concatenar los diferentes archivos. En las columnas de temperatura máxima y mínima se eliminó texto complementario para convertir los valores a decimales float64.
- **Reducción de variables y limpieza de textos:** Se descartaron las columnas de viento y precipitaciones por su alto volumen de nulos (~ 20%) y su menor impacto en el consumo frente a la temperatura.
- **Agrupación de estaciones:** Como el dataset original venía informando con datos de cada estación local, se agruparon los registros por **fecha\_cruce** calculando la media nacional diaria (**.mean()**), reduciendo así el conjunto de datos a **4.372 filas** (una por día).  
Finalmente nos quedamos únicamente con las columnas **fecha\_cruce**, **temperatura\_maxima** y **temperatura\_minima**

| fecha_cruce | temperatura_maxima | temperatura_minima |
|-------------|--------------------|--------------------|
| 2014-01-01  | 12.661988          | 5.972368           |
| 2014-01-02  | 13.752493          | 8.764956           |
| 2014-01-03  | 14.963504          | 9.397372           |
| 2014-01-04  | 13.499846          | 5.309846           |
| 2014-01-05  | 12.584919          | 4.452709           |

#### 4.2.3. Dataset de Festivos y Calendario Laboral

La preparación de este conjunto de datos, compuesto por 424 registros anuales en formato JSON generados previamente mediante la función de Python que consulta la API pública de **Nager.Date**, fue más sencilla, ya que apenas había errores o discordancias en los datos:

- **Homogeneización de columnas:** Se renombraron las variables principales para asegurar la coincidencia con la tabla maestra, transformando **date** en **fecha\_cruce**, **localName** en **nombre\_festivo** y **global** en **festivo\_nacional**.
- **Reducción de variables:** Se eliminaron columnas innecesarias para el modelo predictor, como **countryCode**, **counties** o **launchYear**, preservando únicamente las tres variables renombradas en un formato limpio de texto y valores booleanos.

| fecha_cruce | nombre_festivo                    | festivo_nacional |
|-------------|-----------------------------------|------------------|
| 2014-01-01  | Año Nuevo                         | True             |
| 2014-01-06  | Día de Reyes / Epifanía del Señor | True             |
| 2014-02-28  | Día de Andalucía                  | False            |
| 2014-03-01  | Día de les Illes Balears          | False            |
| 2014-03-19  | San José                          | False            |

### 4.3. Integración y Enriquecimiento: Construcción de la Capa Oro

Una vez que limpios los tres conjuntos de datos por separado en la Capa Plata, el siguiente paso fue juntarlos y realizar los últimos retoques para crear la matriz analítica final (**Capa Oro**). Este proceso se dividió en tres partes: el cruce de las tablas por fecha, el arreglo de valores nulos o vacíos y la transformación de los datos en formatos más sencillos de procesar.

#### 4.3.1. Cruce de tablas (Joins)

Para unificar las tres fuentes de información en una sola tabla, usamos la columna **fecha\_cruce** como punto de unión:

- **Cruce de Demanda y Clima:** Se combinó la demanda y el clima usando un **LEFT JOIN**. Elegimos este cruce para mantener todas las filas de la demanda (que registra datos cada 10 minutos) y repetir el dato del clima diario en cada uno de esos tramos.

- **Cruce con Festivos:** Sobre el resultado anterior se hizo otro **LEFT JOIN** con la tabla de festivos. Los días que no eran festivos se llenaron automáticamente con valores nulos. Para solucionarlo, se recurrió a la función **.fillna()** para poner el valor False en la columna **festivo\_nacional** y el texto 'Laborable' en **nombre\_festivo**.

#### 4.3.2. Tratamiento de nulos en las temperaturas

Al hacer los cruces, aparecieron algunos valores nulos en las columnas de temperatura. Para no borrar esas filas de demanda, se aplicó una interpolación lineal mediante la función **.interpolate()**. Esta función calcula la temperatura que falta basándose en los días anteriores y posteriores. Además, usamos **.bfill()** y **.ffill()** para asegurar que no quedara ningún hueco en los extremos de la tabla.

#### 4.3.3. Ingeniería de Variables (Feature Engineering)

Los modelos de Machine Learning funcionan mejor con números que con texto o fechas. Por eso, se transformaron los datos de la siguiente manera:

- **Hora y minuto:** Separamos la hora de texto (por ejemplo, "**14:30:00**") en dos columnas de números enteros **int64** y se prescindió de los segundos, ya que el modelo de machine learning trabaja mejor con números que columnas en formato **date** o **datetime**.
- **Mes y día de la semana:** Convertimos la fecha en un formato de tiempo real y extrajimos el mes (del 1 al 12) para reflejar los cambios de estación, y el día de la semana (del 0 al 6) para diferenciar los días laborables de los fines de semana.
- **Clasificación de festivos:** Para no dejar los nombres de los festivos en formato de texto, se creó una función que asigna un número entero según la importancia del día y se introdujo el resultado en una nueva columna llamada **tipo\_dia**:
  - **0:** Para días laborables normales.
  - **1:** Para festivos estándar o regionales.

- 2: Para festivos nacionales muy importantes donde el consumo cae drásticamente ('Navidad', 'Año Nuevo', 'Día de Reyes', 'Nochebuena' y 'Nochevieja').

Por último, se eliminó la columna **festivo\_nacional**, ya que ya no era necesaria debido a la nueva columna **tipo\_dia**. Por otro lado, se decidió conservar las columnas **fecha\_cruce** y **nombre\_festivo** en el archivo final para permitir el funcionamiento interactivo del dashboard y proporcionar mayor contexto visual dentro de él.

Se realizó un último ajuste debido al apagón eléctrico sufrido en la península el día 28 de Abril de 2025. Al tratarse de una anomalía, se eliminaron los registros referentes a ese día para evitar sesgos en el modelo predictivo.

| fecha_cruce | hora | minuto | nombre_festivo | temperatura_maxima | temperatura_minima | demanda_mw | fecha      | mes | dia_semana | tipo_dia |
|-------------|------|--------|----------------|--------------------|--------------------|------------|------------|-----|------------|----------|
| 2014-01-01  | 0    | 0      | Año Nuevo      | 12.661988          | 5.972368           | 24546.0    | 2014-01-01 | 1   | 2          | 2        |
| 2014-01-01  | 0    | 10     | Año Nuevo      | 12.661988          | 5.972368           | 24309.0    | 2014-01-01 | 1   | 2          | 2        |
| 2014-01-01  | 0    | 20     | Año Nuevo      | 12.661988          | 5.972368           | 24348.0    | 2014-01-01 | 1   | 2          | 2        |
| 2014-01-01  | 0    | 30     | Año Nuevo      | 12.661988          | 5.972368           | 24321.0    | 2014-01-01 | 1   | 2          | 2        |
| 2014-01-01  | 0    | 40     | Año Nuevo      | 12.661988          | 5.972368           | 24194.0    | 2014-01-01 | 1   | 2          | 2        |

#### 4.4. Persistencia y Almacenamiento Distribuido (HDFS y Parquet)

Como último paso de la Fase 3, se guardó el dataframe de la Capa Oro en un conjunto de ficheros en formato **.parquet**. Para ello, se tomaron dos decisiones para cómo almacenar la información:

- **Uso del formato Parquet particionado:** En lugar de guardar los datos en un archivo CSV tradicional (que ocuparía mucho espacio y sería muy lento de abrir), se exportó el dataframe en formato **Parquet** dividiendo el archivo por año y mes. Esto hace que la información se guarde organizada en carpetas, por lo que, si en el futuro el cuadro de mando necesita consultar una fecha concreta, no tiene que cargar todo el histórico de golpe, sino que va directo a la carpeta donde se encuentra la información que necesita y la carga mucho más rápido.

- **Despliegue en contenedor HDFS (Docker):** Para simular cómo se trabaja con grandes volúmenes en Big Data, se creó un archivo **`docker-compose.yml`** para levantar un clúster local de Hadoop (HDFS) dentro de un contenedor Docker. Una vez encendido el contenedor, se transfieren los ficheros Parquet desde el sistema local hacia la ruta del docker **`/data/gold/dataset_demanda_parquet/`**.

## 5. FASE 6: DESPLIEGUE

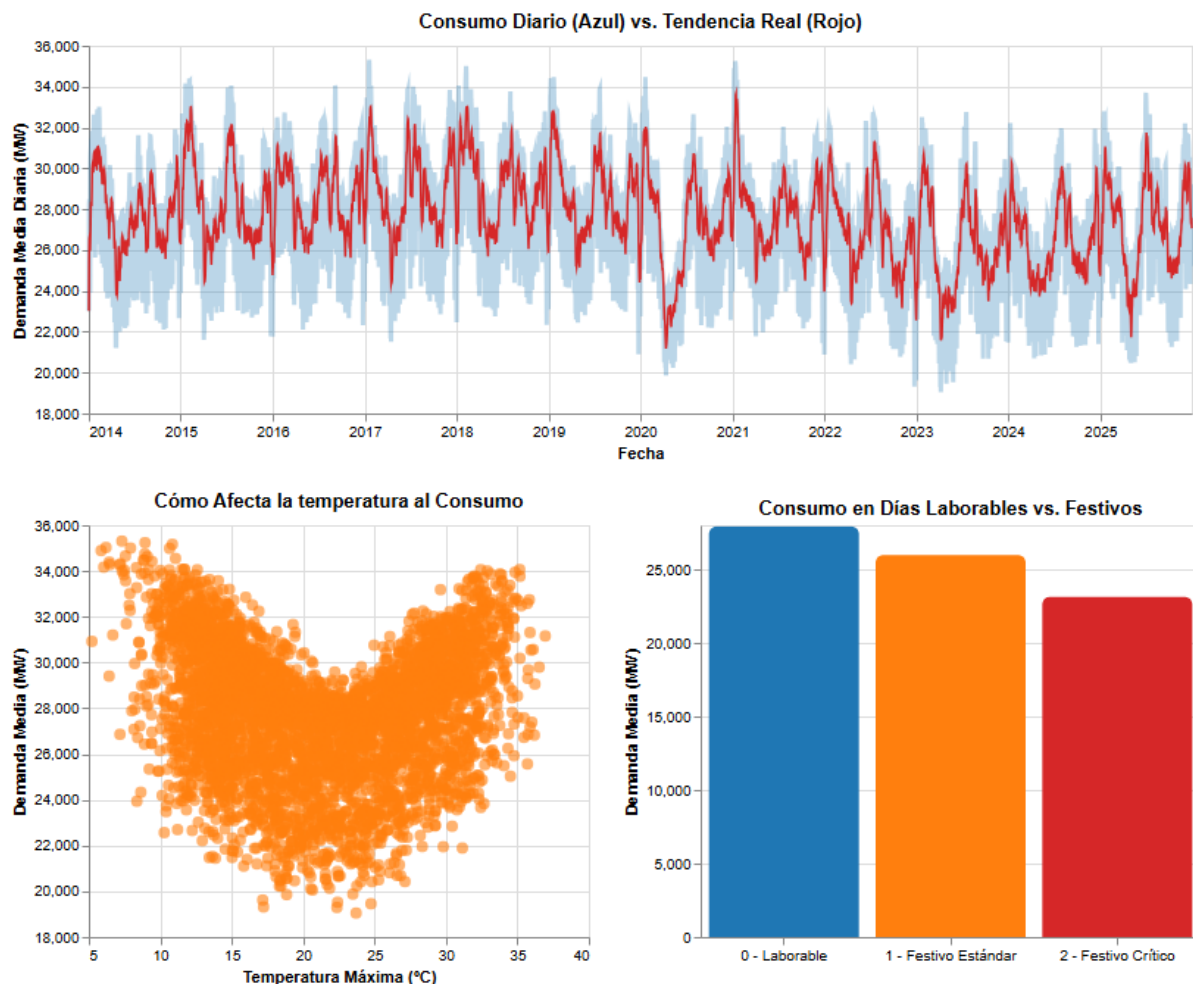
Como las fases de modelado y evaluación se salen del alcance de este módulo de Big Data, el proyecto termina con la fase de despliegue. Al habernos saltado las fases 4 y 5, el despliegue consistirá en mostrar los datos limpios de la Capa Oro a través de un *Dashboard* interactivo.

Para ello, se ha creado una interfaz visual dentro de un cuaderno de Jupyter usando la librería gráfica **Altair**. El diseño está pensado para que los datos se entiendan a golpe de vista, y se han dividido en tres gráficos:

- **Consumo Diario vs. Tendencia Real:** Este gráfico refleja el consumo exacto de cada día (en azul) junto a una media de los últimos 7 días (en rojo), lo que ayuda a ver claramente cómo cambia la demanda según la época del año.
- **Impacto de la Temperatura:** Un gráfico de puntos que muestra de forma visual cómo el consumo forma una "U", demostrando que la demanda de energía sube conforme hace más frío (calefacciones) y conforme hace calor (aire acondicionado, ventiladores, etc).
- **Consumo Laborales vs. Festivos:** Un gráfico que compara cuánto baja la demanda en los días festivos frente a los días de trabajo habituales.

Los gráficos están conectados entre sí de forma interactiva. Si se selecciona un periodo de tiempo concreto en el primer gráfico arrastrando el ratón, los dos

gráficos de abajo se actualizan al instante para mostrar únicamente las medias y temperaturas de esos días seleccionados.



## 6.1. Conclusiones del Análisis de los Datos

Una vez puesto en marcha el dashboard, la información de la Capa Oro nos permite extraer tres conclusiones claras sobre cómo funciona el consumo eléctrico en el país:

1. **El ritmo según la época y la semana:** Los picos más altos de demanda ocurren en invierno y verano, mientras que en primavera y otoño la demanda se estabiliza. Además, el gráfico azul demuestra que el consumo cae cada fin de semana y se recupera de golpe los lunes.

2. **La importancia de la temperatura:** El gráfico del clima confirma que la relación entre temperatura y consumo es una "U". El consumo se mantiene equilibrado cuando hace una temperatura media de unos 20°C. Si baja de ahí, la demanda sube por las calefacciones; pero si sube de los 25 °C, la demanda se dispara de forma mucho más rápida debido al uso masivo de aires acondicionados.
3. **Los festivos:** No todos los días no laborables son iguales. Mientras que un sábado o domingo normal se reduce el consumo de forma moderada, los festivos importantes como Navidad o Año Nuevo provocan una bajada total de la demanda. Esto ocurre porque la industria y el comercio se detienen casi por completo, dejando el consumo del país en su mínimo absoluto.

## 6. DICCIONARIO DE DATOS DE LA CAPA ORO

| Nombre de la Columna | Tipo de Dato          | Qué significa (Descripción)                                                                                  |
|----------------------|-----------------------|--------------------------------------------------------------------------------------------------------------|
| fecha_cruce          | Texto<br>(AAAA-MM-DD) | La fecha del registro. Sirve como "puente" para poder unir todas las tablas.                                 |
| demanda_mw           | Decimal<br>(float64)  | Cantidad de energía eléctrica real consumida en la península durante ese tramo de tiempo.                    |
| temperatura_maxima   | Decimal<br>(float64)  | La temperatura más alta registrada en el país durante ese día (en °C).                                       |
| temperatura_minima   | Decimal<br>(float64)  | La temperatura más baja registrada en el país durante ese día (en °C).                                       |
| hora / minuto        | Entero<br>(int64)     | La hora y el minuto por separado. Esto ayuda a saber en qué momentos exactos del día sube o baja el consumo. |
| mes                  | Entero<br>(int64)     | El mes del año (del 1 al 12). Es clave para ver los cambios según la estación (invierno, verano, etc.).      |

|                       |                   |                                                                                                                                                         |
|-----------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>dia_semana</b>     | Entero<br>(int64) | El día de la semana (donde 0 es lunes y 6 es domingo). Sirve para diferenciar los días de diario de los fines de semana.                                |
| <b>tipo_dia</b>       | Entero<br>(int64) | Número de control diario: 0 para días laborables, 1 para festivos estándar y 2 para festivos muy importantes.                                           |
| <b>nombre_festivo</b> | Texto             | El nombre oficial del festivo (ej. "Navidad"). Si es un día de trabajo normal, se indica como "Laborable".                                              |
| <b>anio</b>           | Entero<br>(int64) | El año del registro. Es el dato que el sistema utiliza para organizar y dividir el archivo final por carpetas. Incluido solo en la tabla del dashboard. |